HireAssist

Project Proposal · Software Architecture Term Project

Project Name

HireAssist — an AI-assisted hiring-support layer for companies that receive more applications than they can screen carefully and interview well.

It is **not** a job board, **not** a job-application platform, and **not** another applicant-tracking system. It attaches to the hiring channels a company already uses and supplies the one thing they lack: the capacity to judge every application properly.

The human still decides. HireAssist makes sure that when they decide, they are deciding on evidence they actually had time to look at.

Group Members

#	Student ID	Name
1	6772055221	Patiphon Puntusin
2	6872036421	Thanabul Parodom
3	6870406621	Thanwarat Korcharoenkiat
4	6870235621	Rerngrit Jangsri

Problem Description

> **บริษัทขนาดเล็กที่มีพนักงานไม่เพียงพอ ไม่สามารถ screen resume ได้ทัน**
>
> *Small companies with insufficient staff cannot screen incoming resumes fast enough.*

Hiring is bursty. A position goes up on JobsDB, JobThai or LinkedIn and applications arrive in a concentrated wave into an inbox or a spreadsheet. Every one must be opened, read and judged by hand, and each surviving candidate interviewed by someone who has actually studied their resume against the role. The people doing that work are rarely dedicated to it — a single HR generalist who also owns payroll and onboarding, or a founder hiring between their actual job.

The decisive factor is not headcount but a **ratio**: applications arriving against the capacity to judge them properly. Screening effort scales with applicant volume; screening capacity scales with headcount, and headcount never keeps up. One recruiter with 80 applicants and five recruiters with 2,000 have the same problem.

When that ratio breaks, quality collapses in two places:

- **Screening becomes shallow** — resumes get a 20-second skim or a crude keyword filter, so good candidates are missed and weak ones advance.
- **Interviews become unprepared** — the interviewer opens the resume minutes beforehand and falls back on generic questions, so the hour fails to verify the very things screening could not confirm.

What that costs the company

- **Lost candidates** — strong applicants are hired by a faster competitor, and the rest are ghosted — which damages employer brand in a small, highly-networked local market.
- **Decisions with no trail** — rejections live in someone's head, so nobody can answer why a candidate was passed over, or check whether screening is consistent.
- **A dead archive and rotting positions** — past applicants are never revisited when a similar role reopens, and a requisition open for 90 days looks exactly like one opened last week.
- **PDPA exposure** — resumes are personal data. Keeping them indefinitely in an inbox with no retention limit is a compliance risk SMEs are least equipped to manage.

Why the existing options do not solve it

- **Enterprise ATS** — built to track applications, not to judge them — and priced for companies with a recruitment department.
- **Job boards** — they solve candidate supply, and in doing so make this problem worse: more resumes into the same inbox.
- **Keyword filters and CV parsers** — mechanical. They match strings rather than weighing evidence, cannot explain a decision, and reject good candidates for using different vocabulary.
- **Spreadsheets and inbox** — the current reality for most SMEs, and exactly the thing that is failing.

The gap is not candidate supply and not record-keeping. It is **judgement capacity** after the applications arrive — which is what HireAssist supplies: explainable screening at volume, interview questions grounded in the specific resume and role, re-matching of the existing talent pool, visibility into stalling positions, and candidate data kept compliant.

Target Customers

> **Any company that receives more applications than it can screen carefully and interview well.**

The customer is defined by a ratio, not by company size. A 20-person startup with one HR generalist and 80 applicants per opening, and a 300-person company with three recruiters and 2,000, are the same customer. A company qualifies when application volume exceeds screening capacity; when the role needs judgement rather than a checkbox filter; when interviews are not systematically prepared; when similar roles recur, so past applicants keep their value; and when hiring runs on an inbox and spreadsheets rather than an adequate ATS.

Being small is not enough to qualify. A 15-person company hiring one person a year has no throughput problem.

Primary segment (initial focus)

Thai technology companies — software houses, product startups, digital agencies and IT service firms, roughly 10 to 200 employees.

Attribute	Profile
Hiring pattern	Bursty and urgent; several roles at once, and the same roles reopen every few months
Volume	~20–200 applications per opening, in concentrated waves after a posting
Capacity	0–2 HR generalists, often helped by a tech lead hiring between their actual work

Existing tools	Inbox, Google Sheets, LINE, a job board or two; rarely a real ATS
Languages	Mixed Thai and English resumes; titles and technical skills mostly English

Why this segment first: tech resumes are comparatively structured, which makes explainable matching tractable; technical roles are exactly where a generic interview wastes the hour; and roles recur, so the talent pool pays off from the first re-match. This is a beachhead, not a boundary — nothing in the design assumes a small company.

Users of the system

User	Goal	What they get
Recruiter / HR	Clear the backlog, shortlist candidates who can be defended, interview them well	A ranked, explainable shortlist instead of a folder of unread PDFs; questions grounded in that resume and role; alerts when a position stalls
Admin / owner	Keep hiring compliant, consistent and auditable, and control access	Member and role management, retention settings, and a recorded reason behind every decision
Candidate (indirect)	Be judged on evidence, and have their data kept no longer than needed	A real reading rather than a 20-second skim; data that ages out on a defined schedule, and an erasure request that is honoured

Scenario (use-case & description)

> **Describe the role → drop in the resumes → get an explained shortlist → walk into the interview prepared → and never lose the candidates you passed on.**

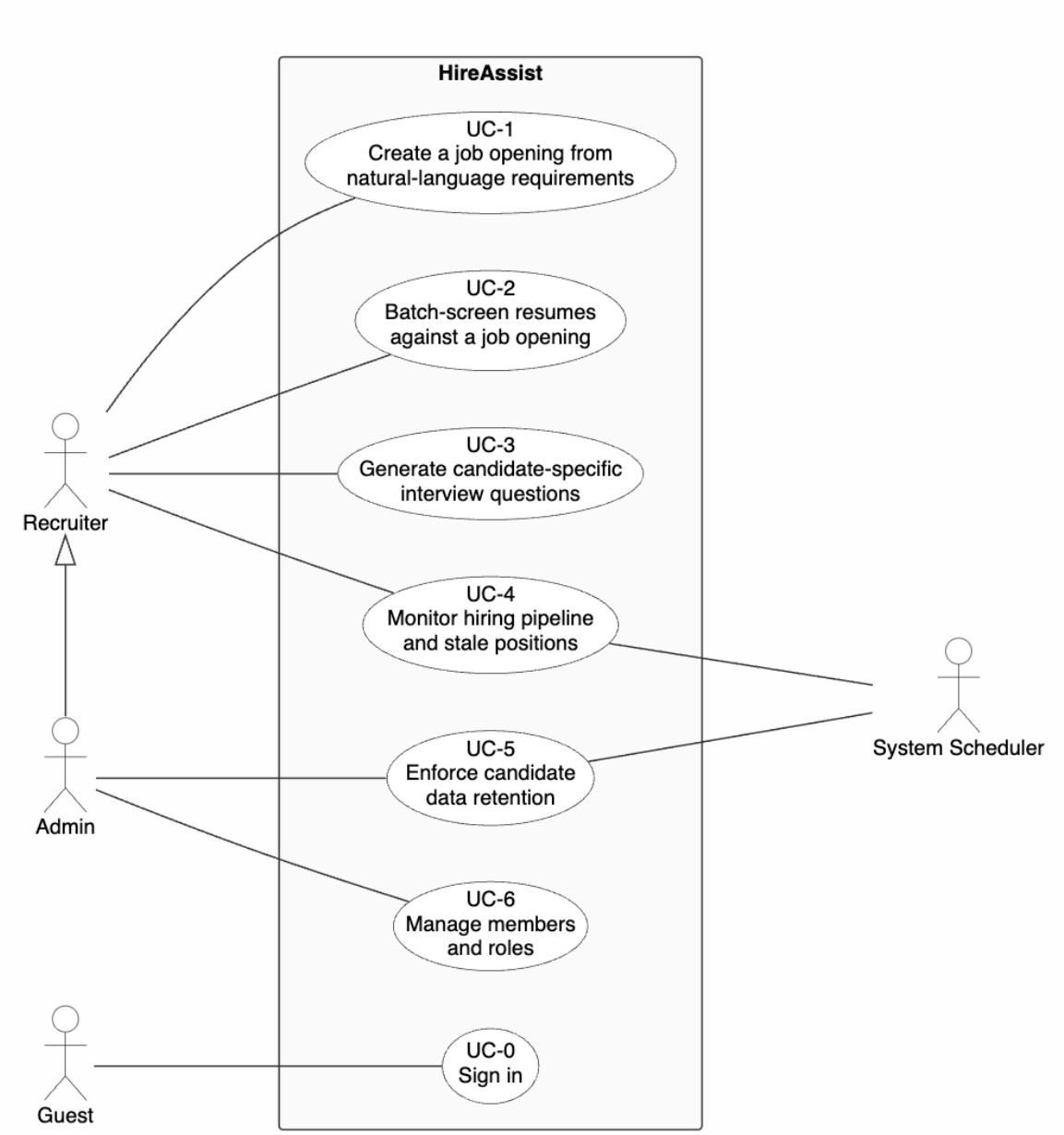
HireAssist is used by a small hiring team through one recruiter-facing web application. Seven use cases cover that flow; UC-0 and UC-6 are foundations, UC-1 to UC-5 are the hiring work itself.

ID	Use case	Primary actor	Consequence it addresses
UC-0	Sign in	Guest	— (foundation)
UC-1	Create a job opening from natural-language requirements	Recruiter	Slow time-to-screen
UC-2	Batch-screen resumes against a job opening	Recruiter	Slow time-to-screen, ghosted candidates, no decision trail
UC-3	Generate candidate-specific interview questions	Recruiter	Unprepared, generic interviews
UC-4	Monitor hiring pipeline and stale positions	Recruiter / System Scheduler	Positions rot silently
UC-5	Enforce candidate data retention	System Scheduler / Admin	PDPA exposure
UC-6	Manage members and roles	Admin	— (foundation)

Actors

Actor	Description
Guest	Anyone who has reached HireAssist but has not signed in. On signing in they act as a Recruiter or an Admin, per the role their account holds.
Recruiter	A signed-in member running day-to-day hiring — an HR generalist, a founder, or a tech lead hiring for their own team. Primary actor of UC-1 to UC-4.
Admin	The signed-in owner of the installation. Every Recruiter capability, plus the retention policy (UC-5) and members and roles (UC-6).
Candidate	Indirect actor. Does not log in and has no interface. Supplies the resume and is the subject of the data processed; an erasure request arrives out of band and an Admin executes it.
System Scheduler	Supporting actor. Time-driven trigger for work no human initiates: staleness checks (UC-4) and retention enforcement (UC-5).

Use case diagram



UML use case diagram.

The use cases

UC-0 — Sign in

Actor: Guest · **Goal:** *Enter the system with the permissions of their role.*

1. Guest signs in with email and password. **2.** System authenticates and issues a session token carrying their role. **3.** The user acts as a Recruiter or an Admin accordingly. **4.** Every later request is authorised at the gateway against that token.

UC-1 — Create a job opening from natural-language requirements

Actor: Recruiter · **Goal:** *Turn the way a role is actually described in conversation into structured, machine-usable criteria — without filling in a long form.*

1. Recruiter pastes a free-text description, Thai or English. **2.** System extracts a proposed title and a weighted, categorised criteria set. **3.** Recruiter reviews and edits criteria, weights and must-have flags. **4.** Recruiter sets expected time-to-fill and confirms. **5.** System stores the opening as open and reports it to the pipeline dashboard.

UC-2 — Batch-screen resumes against a job opening

Actor: Recruiter · **Goal:** *Convert a pile of unread resumes into a ranked shortlist with a stated reason for every ranking.*

1. Recruiter uploads a batch of PDF resumes against an open job opening. **2.** System validates the files, creates a screening batch and returns a batch ID immediately. **3.** System parses each resume into a normalised candidate profile. **4.** System scores each profile against the criteria and writes a justification. **5.** Results stream into the ranked shortlist as they complete. **6.** Recruiter overrides any score the AI got wrong and shortlists candidates. **7.** System records each decision and adds every candidate to the talent pool with the date their data was collected.

UC-3 — Generate candidate-specific interview questions

Actor: Recruiter · **Goal:** *Walk into an interview with questions grounded in this candidate's resume and this job's requirements.*

1. Recruiter opens a shortlisted candidate and requests an interview guide. **2.** System retrieves the candidate profile, the resume and the job criteria. **3.** System generates questions covering resume claims, must-have gaps and role depth, each tagged with its source criterion. **4.** Recruiter edits, removes, adds or regenerates questions. **5.** System retains the guide for retrieval before the interview.

UC-4 — Monitor hiring pipeline and stale positions

Actor: Recruiter (primary), System Scheduler (supporting) · **Goal:** *See the health of every open position at a glance, and be told when one is stalling.*

1. System aggregates pipeline metrics across all open positions. **2.** Dashboard shows per-position funnels and days open against expected time-to-fill. **3.** Scheduler periodically evaluates every open position against its staleness rule. **4.** Breaching positions are flagged, an alert is published, and the responsible Recruiter is notified. **5.** The flag clears when the position is closed or the condition is resolved.

UC-5 — Enforce candidate data retention

Actor: System Scheduler (primary), Admin (configures the policy) · **Goal:** *Make sure candidate data is not kept longer than the company is entitled to keep it — without anyone remembering to check.*

1. Admin configures the retention period, the anchor date and the expiry action. **2.** Scheduler periodically evaluates every candidate record against the policy. **3.** Records entering the warning window are flagged and the Recruiter is notified with the expiry date. **4.** On expiry the system deletes or anonymises the record and everything derived from it, including stored resume files. **5.** System writes an audit entry and removes the candidate from the talent pool.

UC-6 — Manage members and roles

Actor: Admin · **Goal:** *Control who has access and what each member may do.*

1. Admin creates a member account, removes a member, or changes a member's role. **2.** For a new member, Admin sets email, an initial password and a role. **3.** System applies the change to the membership list. **4.** Admin passes the credentials to the new member outside the system.

Deferred

D-1 — Re-match the talent pool against a job opening. Score candidates the company has already seen against a new opening, instead of sourcing from zero. Deferred: it cannot be demonstrated without a deep talent pool, which this project will not have. It is a scope decision, not an omission — the retention and talent-pool requirements that make it possible are already in place.

Functional Requirements

44 functional requirements, each a single verifiable statement carrying no design or technology choice. Numbered FR-<use case>.<n> so every requirement traces to the use case it serves, and so adding one does not renumber the rest. IDs are stable once written and never reused; a withdrawn requirement leaves a gap.

UC-0 — Sign in

ID	Requirement
FR-0.1	The system shall authenticate a Guest by email address and password, and issue a session token identifying the user and their role, under which the user acts as a Recruiter or an Admin.
FR-0.2	The system shall reject any request presenting a missing, invalid, or expired session token.
FR-0.3	The system shall support two roles, Admin and Recruiter, where Admin holds all Recruiter permissions in addition to its own.
FR-0.4	The system shall reject any request requiring a role higher than the requester's assigned role, and record the rejected attempt.

UC-1 — Create a job opening from natural-language requirements

ID	Requirement
FR-1.1	The system shall accept a free-text role description in Thai or English as the input to job opening creation.
FR-1.2	The system shall derive from that description a proposed job title and a set of screening criteria, each classified as must-have or nice-to-have and assigned a weight.
FR-1.3	The system shall present the derived proposal to the Recruiter for review before the job opening is created.
FR-1.4	The system shall allow the Recruiter to add, edit or remove any criterion and change its classification and weight before confirmation.
FR-1.5	The system shall identify the parts of a description it could not interpret, and shall allow criteria to be entered manually either to fill those gaps or to replace the derived proposal entirely.
FR-1.6	The system shall record an expected time-to-fill for each job opening.
FR-1.7	The system shall store a confirmed job opening in the open state, recording the date it was opened and retaining the original free-text description.
FR-1.8	The system shall allow a Recruiter or Admin to pause and resume a job opening, and to close it with a recorded outcome of either *filled* or *cancelled*.

UC-2 — Batch-screen resumes against a job opening

ID	Requirement
FR-2.1	The system shall accept a batch of resume files in PDF format submitted against one open job opening.
FR-2.2	The system shall acknowledge a submitted batch with a batch identifier without waiting for processing of that batch to complete.
FR-2.3	The system shall parse each resume into a normalised candidate profile.
FR-2.4	The system shall process each resume in a batch independently, such that failure of one resume does not prevent processing of the others.
FR-2.5	The system shall produce for each resume a screening result comprising a score against the job opening's weighted criteria, a must-have check stating which mandatory criteria are met and which are not, and a written justification citing evidence drawn from that resume. Where any must-have criterion is not met, the system shall mark the candidate not qualified and record which criteria caused it.

FR-2.6	The system shall present screening results ranked by score, updating the ranking as further results become available, and shall allow the list to be filtered by minimum score and by individual criterion.
FR-2.7	The system shall retry failed processing attempts and shall flag any resume it cannot parse or score as requiring manual review, excluding it from the ranking rather than assigning it a score.
FR-2.8	The system shall allow the Recruiter to override a screening score, and shall retain the original score alongside the override.
FR-2.9	The system shall allow the Recruiter to shortlist a screened candidate. A candidate who is neither shortlisted nor marked not qualified remains undecided; the system shall not require a rejection to be recorded by hand.
FR-2.10	The system shall not transmit a rejection reason to the candidate it concerns.
FR-2.11	The system shall add every screened candidate to the talent pool, recording the date their personal data was collected.
FR-2.12	The system shall notify the Recruiter when a screening batch has finished processing.

UC-3 — Generate candidate-specific interview questions

ID	Requirement
FR-3.1	The system shall generate an interview guide for a candidate shortlisted against a job opening, derived from both the candidate's profile and the job opening's criteria, and shall retain the guide so that it can be retrieved again before the interview.
FR-3.2	The system shall include questions probing claims made in the resume, questions addressing gaps against must-have criteria, and questions covering role-specific depth.
FR-3.3	The system shall tag each generated question with the criterion it tests and the resume evidence it was derived from.
FR-3.4	The system shall allow the Recruiter to add, edit, remove or regenerate questions, including regenerating a guide with a stated emphasis.
FR-3.5	The system shall allow interview notes to be recorded against each question in a guide.

UC-4 — Monitor hiring pipeline and stale positions

ID	Requirement
FR-4.1	The system shall present, for each open job opening, the number of candidates at each pipeline stage (applied, screened, shortlisted), days open against its expected time-to-fill, and whether the opening is currently flagged.
FR-4.2	The system shall evaluate every open job opening against its staleness rule on a recurring schedule, independently of whether any user views the dashboard.
FR-4.3	The system shall flag a job opening that has been open longer than its expected time-to-fill.
FR-4.4	The system shall notify the Recruiter responsible for a job opening when that opening is flagged.
FR-4.5	The system shall clear a flag when the job opening is closed, or when the flagged condition is resolved.
FR-4.6	The system shall exclude a paused job opening from staleness evaluation.

UC-5 — Enforce candidate data retention

ID	Requirement
FR-5.1	The system shall allow an Admin to configure a retention period, an anchor date of either the date the data was collected or the date of last activity, and an expiry action of deletion or anonymisation.
FR-5.2	The system shall evaluate candidate records against the retention policy on a recurring schedule, flagging

	those entering the warning window before expiry and notifying the responsible Recruiter with the affected candidates and their expiry date.
FR-5.3	The system shall, on expiry, delete or anonymise everywhere it is held the candidate's personal data — the candidate profile, the stored resume file, screening results and their justification text, and generated interview guides — according to the configured expiry action, as a single operation that leaves no partially erased record in place.
FR-5.4	The system shall, when anonymising, remove all identifying data while preserving the aggregate counts used for pipeline metrics, such that no later processing can surface the person.
FR-5.5	The system shall write an audit entry for each erasure recording what was erased, when, and under which retention policy, excluding from that entry the candidate's name, contact details, resume file and its contents, candidate profile field values, screening scores and justification text; each entry shall reference a candidate only by a pseudonymous internal identifier that cannot be resolved to a person once erasure is complete.
FR-5.6	The system shall hold expiry for a candidate in an active hiring process and flag the record for the Recruiter to decide.
FR-5.7	The system shall allow an Admin to execute an erasure for a named candidate immediately, rather than waiting for the next scheduled evaluation.
FR-5.8	The system shall, where erasure of stored data fails, retain the record, log the failure, retry, and report the record as deletion pending rather than as erased.

UC-6 — Manage members and roles

ID	Requirement
FR-6.1	The system shall allow an Admin to create a member account with an initial password, remove a member, and change a member's role.

Non-functional Requirements

17 non-functional requirements, grouped by the quality attribute each serves and paired with how it is verified. **Scalability is the quality attribute this project demonstrates.**

Every figure is an initial target, not a measured result — they exist to give the architecture direction, and will be revised once load testing produces real numbers. PDPA compliance is not stated as a single requirement because it is not verifiable as one; it is the combined effect of NFR-12 to NFR-14 and the UC-5 retention requirements.

Performance

ID	Requirement	Verified by
NFR-01	The system shall parse a resume into a normalised candidate profile within 20 seconds at the 95th percentile, and shall abandon and retry any attempt exceeding 60 seconds.	Load test
NFR-02	The system shall produce a screening result for one parsed profile within 20 seconds at the 95th percentile, and shall abandon and retry any attempt exceeding 60 seconds.	Load test
NFR-03	The system shall return a ranked candidate list within 2 seconds for a job opening holding up to 1,000 screened candidates.	Load test
NFR-04	The system shall acknowledge a submitted screening batch within 2 seconds, independently of the size of the batch.	Load test
NFR-05	The system shall produce an interview guide within 30 seconds at the 95th percentile, indicating progress while it generates.	Load test

Scalability

ID	Requirement	Verified by
NFR-06	The system shall complete a screening batch of 100 resumes within 10 minutes when at least 4 screening workers are available.	Load test
NFR-07	The system shall increase batch screening throughput proportionally as screening workers are added, up to at least 4 workers.	Comparative load test
NFR-08	The system shall sustain 20 concurrent Recruiters, and batches of up to 200 resume files of up to 10 MB each, without breaching NFR-01 to NFR-06.	Load test

Availability & Reliability

ID	Requirement	Verified by
NFR-09	The system shall be available at least 99% of the time Monday to Friday, 08:00–20:00 ICT.	Monitoring
NFR-10	The system shall not lose a submitted resume: every resume in an accepted batch shall reach a terminal state — scored, or flagged for manual review — even across a restart of any single service.	Fault-injection test

Security

ID	Requirement	Verified by
NFR-11	The system shall encrypt candidate personal data at rest and shall transmit all data over TLS 1.2 or higher.	Configuration review
NFR-12	The system shall record an audit log of every access to candidate personal data, and shall retain that log for at least one year.	Inspection

Privacy & Regulatory Compliance

ID	Requirement	Verified by
NFR-13	The system shall complete a valid candidate erasure request within 30 days of receiving it.	Audit log inspection
NFR-14	The system shall exclude gender, age, marital status, religion, nationality and photograph from any input used to compute a match score.	Code and prompt review; paired-resume test

Transparency

ID	Requirement	Verified by
NFR-15	The system shall present, for every match score, the per-criterion evidence that produced it, such that a Recruiter can verify the result without reading the resume.	Inspection

Usability & Localisation

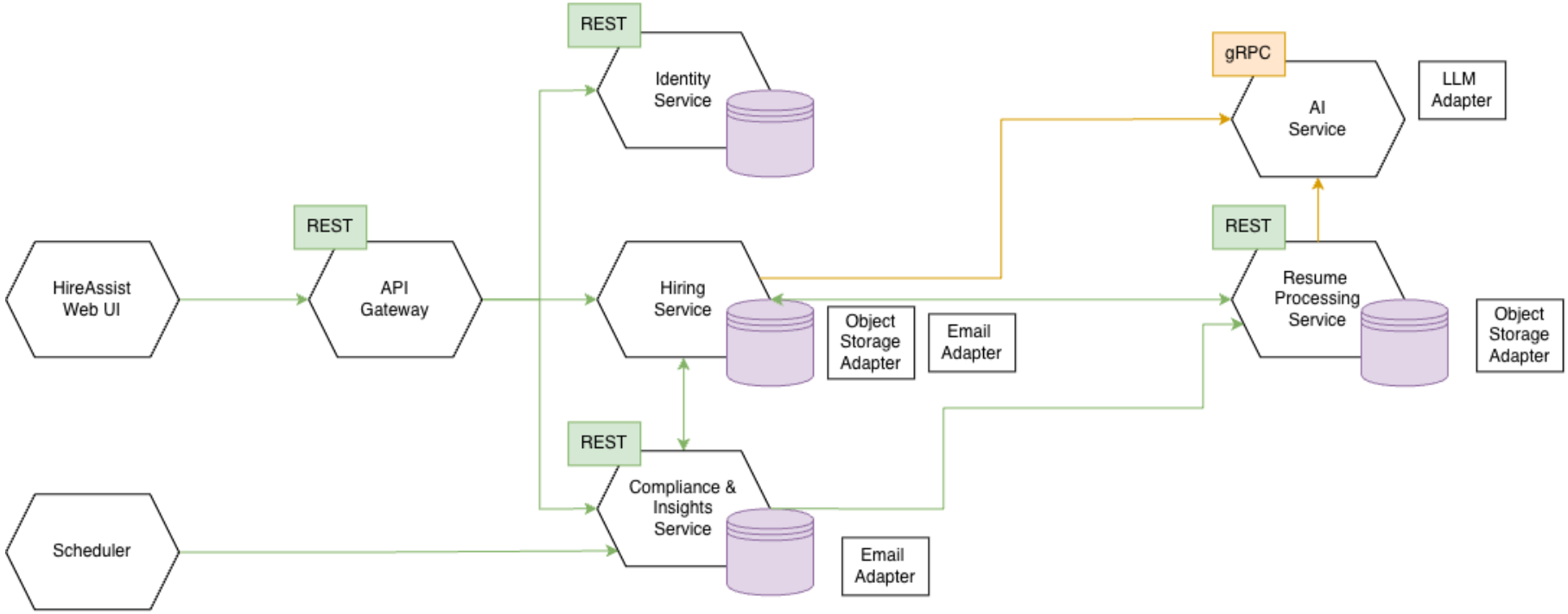
ID	Requirement	Verified by
NFR-16	The system shall provide the user interface in Thai and English, usable on desktop and tablet in current versions of Chrome, Safari and Edge.	Manual test

Modifiability

ID	Requirement	Verified by
NFR-17	The system shall allow the scoring model or prompt to be changed without modifying or redeploying any service other than the one that performs scoring.	Design review

Microservice Design with Collaborations

Five services behind one API gateway. The diagram shows who calls whom; the table below lists every operation each service exposes and every operation it calls on another. The two describe the same system and must agree.



An arrow A → B means "A calls B"; responses are not drawn. A service's private store is its own — no service reads another's, it calls the owning service instead. An adapter box stands for an external system the service reaches only through it. Every call is REST over HTTP/JSON except the two into the AI Service, which are gRPC. There is no message broker.

Service – Operations – Collaborators

Service	Operations	Collaborators

Identity Service
 	
owns: accounts, memberships, roles, sessions
 	
UC-0 · UC-6
	`authenticateMember()`
	`validateSession()`
	`getMemberRole()`
	`createMember()`
	`removeMember()`
	`changeMemberRole()`
		(none)

Hiring Service
 	
owns: job openings, criteria, screening batches and entry status,
	`createJobOpening()`
	`reviseScreeningCriteria()`
	`getScreeningCriteria()`
	`getJobOpening()`
	`pauseOpening()`
		Object Storage Adapter
		`storeResumeFile()`
		Resume Processing Service
		`submitResumeForScreening()`
		(carries the criteria snapshot)

Service	Operations	Collaborators

scores and overrides, shortlist decisions, interview guides
 	
UC-1 · UC-2 · UC-3
	`resumeOpening()`
	`closeOpening()`
	`submitScreeningBatch()`
	`getBatchStatus()`
	`getRankedResults()`
	`overrideScore()`
	`shortlistCandidate()`
	`generateInterviewGuide()`
	`reviseGuide()`
	`getInterviewGuide()`
	`recordInterviewNote()`
	`reportScreeningResult()`
	`eraseHiringData()`
		`getCandidateProfile()`
		AI Service *(gRPC)*
		`deriveCriteriaFromDescription()`
		`generateInterviewQuestions()`
		Compliance & Insights Service
		`recordPipelineEvent()`
		Email Adapter
		`sendEmail()`

Resume Processing Service
 	
owns: candidate identity and collection date, candidate profiles, parsed resume text, per-criterion scoring output and justifications
 	
UC-2 (per-resume work)
	`submitResumeForScreening()`
	`getCandidateProfile()`
	`listCandidateProfiles()`
	`listUnparsableResumes()`
	`eraseCandidateProfile()`
		Object Storage Adapter
		`fetchResumeFile()`
		AI Service *(gRPC)*
		`extractProfileFromText()`
		`scoreAgainstCriteria()`
		Hiring Service
		`reportScreeningResult()` *(callback)*

AI Service
 	
owns: prompts and model credentials — no domain data
 	
UC-1 · UC-2 · UC-3
	`deriveCriteriaFromDescription()`
	`extractProfileFromText()`
	`scoreAgainstCriteria()`
	`generateInterviewQuestions()`
		LLM Adapter
		`complete()`

Compliance & Insights Service
 	
owns: retention policy, pipeline metrics, erasure and access audit logs
 	
UC-4 · UC-5
	`setRetentionPolicy()`
	`evaluateRetention()`
	`listHeldExpiries()`
	`resolveHeldExpiry()`
	`eraseCandidate()`
	`getErasureAudit()`
	`getPipelineDashboard()`
	`evaluateStaleness()`
	`recordPipelineEvent()`
	`recordAuthorisationRejected()`
		Hiring Service
		`eraseHiringData()`
		Resume Processing Service
		`eraseCandidateProfile()`
		`listCandidateProfiles()`
		Email Adapter
		`sendEmail()`

The API Gateway is the entry point, not a domain service: it terminates TLS, routes, rate-limits, resolves the caller's role, validates the session through the Identity Service, and records a rejected request to the access audit log. It owns no data. External systems — object storage, the model provider, the email provider — are each reached only through an adapter. No service reads another service's data store; every crossing is one of the operations above.